

③ JavaScript

⇒ To print something -

Code -

```
<script>
```

```
  < document.write (" Hello World");
```

```
</script>
```

Output -

Hello World

≡ for linking javascript to html

```
<script src="main.js"></script>
```

≡ Comments -

```
/* ..... */
```

OR //

Built-in functions -

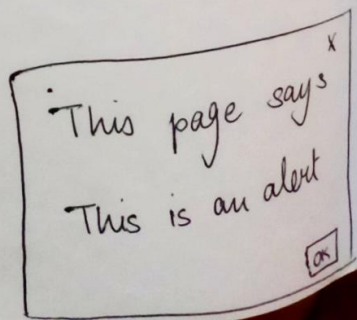
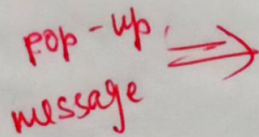
1) ~~script~~ alert (); - It blocks the rendering that is happening on the page.

Code -

```
alert ("This is an alert");
```

Output -

Pop-up
message



(2) Console log() -

↓ class ↓ method

code- console.log("This is a log");

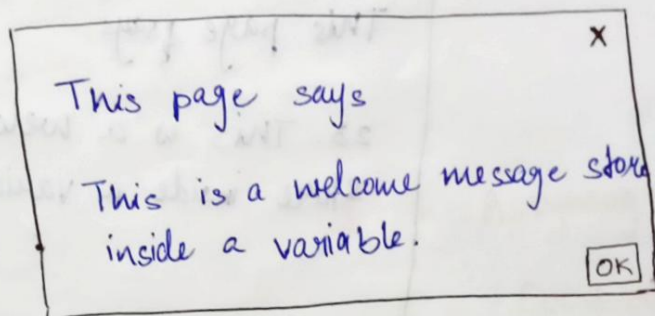
→ Output will be generated on console.

3 var welcome_msg = "This is a welcome message";

identifier → store inside a variable; → string

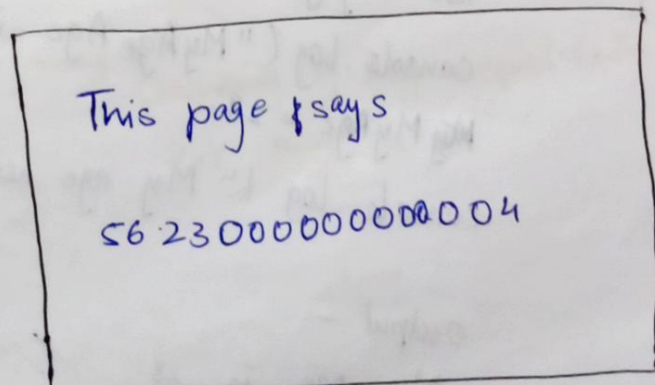
alert (welcome-msg);

Output -



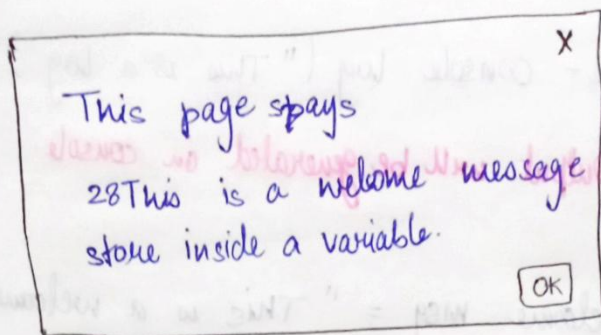
```
⇒ var myAge = 28;  
   var exactAge = 28.23;  
   alert (myAge + exactAge);
```

Output -



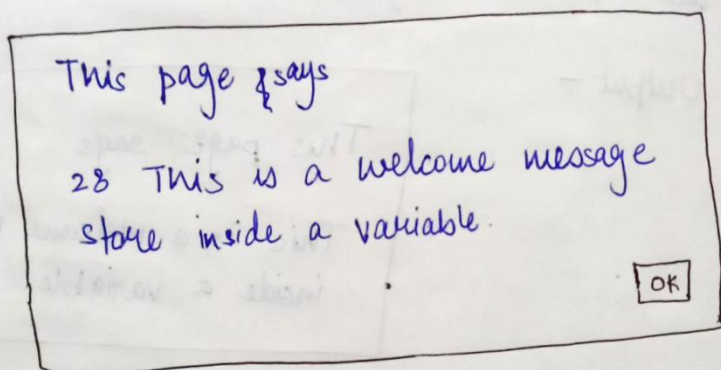
⇒ Alert (myAge + welcome - msg)

Output -



→ Alert (myAge, welcome - msg)

Output -



4 Reassigning variables -

code -

```
var MyAge = 28;
```

```
console.log("My Age is" + myAge);
```

```
My MyAge = 29;
```

```
console.log("My age next year will be" + MyAge);
```

Output -

My Age is 28.

My age next year will be 29.

OR `var MyAge = 28;`
`console.log("....");`
`MyAge = MyAge + 1;`
`console.log("....");`

OR `var MyAge = 28;`
`console.log("....");`
`MyAge++;`
`console.log("....");`

OR `MyAge += 1;`

→ Code -

```
var MyName;  
console.log("My Name is " + MyName);
```

Output -

My Name is undefined.

≤ Boolean Operators

In console -

`My = true`

↙
Assigning my to true

→ `My == true`

Output - true

* = - Assigning

== - Comparing

→ Comparing if My is equal to true

→ `My == "true"`

Output - False

→ Comparing if My is equal to the text true.

→ `My == False`

Output - False

6 Comparing values

→ var isPremiumUser = false

isPremiumUser == true

Output - False

→ 7 < 7

Output - False

→ 7 >= 7

Output - True

→ 5 < 7

Output - False True

7 if statements

code-

```
var ifPremiumUser = false;
```

```
if (isPremiumUser == true) {
```

```
  console.log("Thanks for becoming a premium user");
```

```
} else {
```

```
  console.log("You should subscribe to become a premium user.");
```

```
}
```

Output - You should subscribe to become a premium user.

→ code

```
var myAge = 14;
```

```
if (myAge < 1)
```

```
{ console.log("You are a baby.");
```

```
} else if (myAge < 3) {
```

```

    console.log("You are a toddler.");
  } else if (myAge < 10) {
    console.log("You are a kid.");
  } else if (myAge < 20) {
    console.log("You are a teenager.");
  } else if (myAge < 40) {
    console.log("You are an adult.");
  } else {
    console.log("You must be old. you are not  
in our records.");
  }
}

```

Output - You are a teenager.

8 For loop -

```

code - for (var i=0 ; i<10 ; i++) {
    console.log("Current i value is", i);
  }

```

Output -

console i value is 0.
 console i value is 1.
 console i value is 2.
 console i value is 3.
 console i value is 4.
 console i value is 5.
 console i value is 6.

console i value is 7.
 console i value is 8.
 console i value is 9.

9 Defining functions -

Code -

```
function myFunction() {  
  console.log("My name is Nick.");  
}  
myFunction();
```

Output - My name is Nick.

```
⇒ function printName(name) {  
  console.log("Hello, ", name);  
}  
printName("Nick.");
```

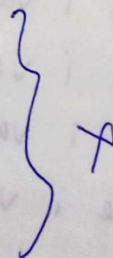
Output - Hello, Nick.

```
⇒ function print(name, age) {  
  console.log("Hello, ", name, " You are ", age);  
}  
print("Nick", 28);  
print("Ansh", 4);
```

10 Event handling -

eg - clicking a button

Code



Code With Harry - JavaScript

⇒ Code -

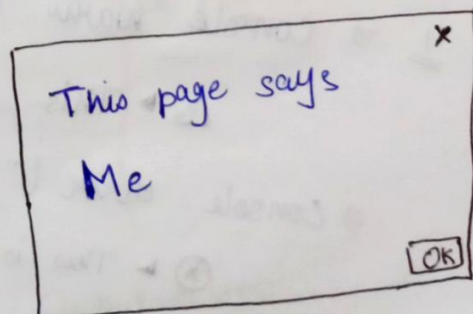
```
<html>
<head>
  <title> JavaScript Tutorial </title>
</head>
<body>
  <h1> Welcome to JavaScript. </h1>
  <div id="FirstContainer" class="container">
    <button id="click"> Click Me </button>
  </div>
  <div class="container">
    <h4> This is your paragraph. </h4>
  </div>
</body>

<script>
  console.log("Hello World!")
  alert("me")
</script>
</html>
```

Output -

Welcome to JavaScript

This is your paragraph.



The console will be cleared

Console -

Hello World!

> First Container . inner HTML = "Hello Harry";

"Hello Harry"

instead of Click Me

Hello Harry will be shown

> document . get Element by TagName ('h1')

▶ HTML Collection [h1]

> document . get Element By TagName ('h1')[0]

<h1> Welcome to ~~this~~ Java Script.

> document . get Element By TagName ('h1')[0] . style . font size = "99" px

CSS can also
be controlled
by JS

Welcome to Java Script

⇒ If we want the text to be displayed on screen instead of console we can write -

document . write ("Hello World!")

1 ⇒ console . warn ("This is a warning")

△ ▶ This is a warning.

⇒ console . error ("This is an error")

⊗ ▶ This is an error

⇒ console . clear()

The console will be cleared.

⇒ console.assert (4 == 6)

⊗ ▶ Assertion failed console.assert

⇒ The assert() method writes a message to the console if an expression evaluates to false.

2 Variables in JavaScript

↓
containers to store data values

⇒ var num1 = 56

var num2 = 40

console.log (num1 + num2);

Output - 96

3 Data types in JavaScript

⇒ var num1 = 455

var num2 = 56.76; } → numbers

⇒ var str1 = "This is a string";

var str2 = "This is also a string"; } → strings

⇒ var marks = {

harvi: 34,

shubham: 78,

harry, 99.977

}

} → object

console.log (num1);

console.log (num2);

Output - 455

Output - 56.76

```
console.log('string 1');  
console.log('string 2');  
console.log('marks');
```

Output - This is a string
Output - This is also a string

Output - { ~~name~~ name: 34,
shubham: 78, harry: 99 } 977

```
→ var a = true;  
var b = false;  
console.log(a, b);
```

Output - true false

} Boolean

```
→ var a = undefined;  
var b;  
console.log(a, b);
```

Output - undefined undefined

} undefined

```
→ var n = null;  
console.log(n);
```

Output - null

} null

2 types of datatypes

Primitive

number null Boolean string symbol undefined

Reference

object array

Arrays -

```
var arr = [1, 2, 'bablu', 4, 5];  
console.log(arr);
```

Output - (5) [1, 2, 'bablu', 4, 5]

console.log(arr[1])

Output - 2

4 Operators in JavaScript

(i) Arithmetic Operators -

var x = 100;

var y = 10;

- + console.log("The value of $x+y$ is ", a+b);
- console.log("The value of $x-y$ is ", a-b);
- * console.log("The value of $x*y$ is ", a*b);
- / console.log("The value of x/y is ", a/b);

Output -

The value of $x+y$ is 110.

The value of $x-y$ is 90.

The value of $x*y$ is 1000.

The value of x/y is 10.

(ii) Assignment Operators

=, +=, -=, *=, /=

var c = 10

console.log(c);

Output - 10

~~var~~ c += 2

console.log(c);

Output = 12

~~var~~ c -= 2

Output = 8

c *= 2

Output = 20

c /= 2

Output = 5

(iii) Comparison Operator -

$=$, $!$, $>$, $<$, $>=$, $<=$

$\Rightarrow$ var m = 34;

var n = 29;

console.log(m == n); false

console.log(m != n); true

console.log(m >= n); true

console.log(m <= n); false

console.log(m > n); true

console.log(m < n); false

(iv) Logical Operators -

&&, ||, !

AND - console.log(true && true); True

console.log(true && false); False

console.log(false && true); False

console.log(false && false); False

OR - console.log(true || false); True

console.log(true || true); True

console.log(false || true); True

console.log(false || false); False

NOT - console.log(!true); False

console.log(!false); True

5 Functions in JavaScript

Eg- ~~the~~ calculating average.

```
⇒ function avg(a, b) {  
    var c = (a+b)/2;  
    return c;  
}
```

```
c1 = avg(4, 6);
```

```
c2 = avg(14, 16);
```

```
console.log(c1, c2);
```

Output - 5 15

6 Conditionals in JavaScript

(i) Single -if statement

```
var age = 14
```

```
if (age > 18)
```

```
{ console.log("You can vote");
```

```
}
```

Output -

[Blank]

(ii) If-else statement

```
var age = 14
```

```
if (age > 18)
```

```
{ console.log("You can vote");
```

```
}
```

```
else
```

```
{ console.log("You can not vote");
```

```
}
```

Output - You can not vote

(iii) else - if ladder
var age = 14;
if (age > 60)

```
{  
  console.log("Old");
```

```
}
```

else if (age > 19)

```
{
```

```
  console.log("Adult");
```

```
}
```

else if (age > 12)

```
{
```

```
  console.log("Teenager");
```

```
}
```

else if (age > 3)

```
{
```

```
  console.log("Kid");
```

```
}
```

else ~~if (age > 3)~~

```
{
```

```
  console.log("Baby");
```

```
}
```

Output - Teenager

7 Loops in JavaScript

eg- printing elements in array using for loop

Method 1 -

```
var arr = [1, 2, 3, 4, 5, 6, 7]
for (var i=0, i < arr.length, i++) {
    console.log (arr[i]);
}
```

Output -

1
2
3
4
5
6
7

Method 2 -

```
var arr = [1, 2, 3, 4, 5, 6, 7]
arr.forEach (function (element) {
    console.log (element);
})
```

Output -

1
2
3
4
5
6
7

eg- using while loop

```
var arr = [1, 2, 3, 4, 5]
let j = 0;
while (j < arr.length) {
    console.log (arr[j]);
    j++;
}
```

Output -

- 1
- 2
- 3
- 4
- 5

eg - using do-while

```
var arr = [1, 2, 3, 4, 5]
```

```
let i = 0
```

```
do {
```

```
  console.log(arr[i]);
```

```
  i++;
```

```
} while (i < arr.length)
```

Output -

- 1
- 2
- 3
- 4
- 5

8 Break and Continue statements

```
var arr = [1, 2, 3, 4, 5, 6]
```

```
for (var i = 0; i < arr.length; i++)
```

```
{
```

```
  if (i == 2) {
```

```
    break;
```

```
  }
```

```
  console.log(arr[i]);
```

```
}
```

Output -

- 1
- 2

Continue;

Output -

- 1
- 2
- 4
- 5
- 6

9 Arrays in JavaScript

let myArr = ["Fan", "Camera", 34, null, true];

Array methods -

(i) length() - returns the length of the array

~~syntax~~ ~~myArr.length()~~
console.log(myArr.length);

Output - 5

(ii) Pop() - Removes the last element

myArr.pop();
console.log(myArr);

Output - (4) ['Fan', 'Camera', 34, null]

(iii) Push() - adding a new element at last

myArr.push("Harry");
console.log(myArr);

Output - (5) ['Fan', 'Camera', 34, null, 'Harry']

(iv) Shift() - Remove the first element

myArr.shift();
console.log(myArr);

~~Output~~ Output - (4) ['Camera', 34, null, 'Harry']

(v) toString() - Converts the array to string

myArr.toString();
~~console.log(myArr);~~

Output - 'Harry, camera, 34, , Harry'

(vi) Sort() - Sort the array in alphabetical order

Code -

```
d = [1, 4, 6, 43, 23, 32324]
```

```
d.sort();
```

Output - [1, 4, 23, 32324, 4, 43, 6]

10 strings in JavaScript

Let myLovelyString = "Harry is a good boy";

String methods -

(i) length -

```
console console.log(myLovelyString.length);
```

Output - 19

(ii) indexOf -

```
console.log(myLovelyString.indexOf("good"));
```

Output - 11

(iii) lastIndexOf -

```
console.log(myLovelyString.lastIndexOf("good"));
```

Output - 11

(iv) slice - [start, stop + 1]

```
console.log(myLovelyString.slice(0, 10));
```

Output - Harry is a

(N) Replace

↙ d = myLovelyString.replace("Harry", "Rohan");
console.log(d);

only
replaces the
first occurrence
of a word

Output - Rohan is a good boy.

f myLovelyString = "Rohan is a good boy
good good";

e = myLovelyString.replace("good",
"bad");

Output - Rohan is a bad boy good
good.

11 Dates in JavaScript

let myDate = new Date();

console.log(myDate); Sun Jul 07 2024

console.log(myDate.getTime()); 1720376029066

console.log(myDate.getDay()); 0 (-Sunday)

console.log(myDate.getHours()); 23

console.log(myDate.getFullYear()); 2024

0 - Sunday

1 - Monday

2 - Tuesday

3 - Wednesday

4 - Thursday

5 - Friday

6 - Saturday

12 DOM Manipulation

In Console -

> document -

HTML document appears

> document.location

host, origin, etc

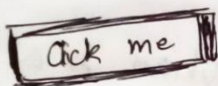
DOM - Document Object Module

> document.getElementById('click').click()

undefined.

> document.getElementById('click').style.border

= "2px solid blue"



DOM Manipulation Methods

i) document.getElementById -

```
let elem = document.getElementById('click');
```

```
console.log(elem);
```

Output Console - `<button id="click">Click me</button>`

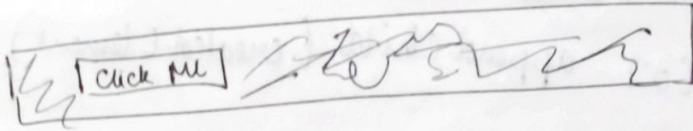
(ii) document.getElementsByClassName -

```
let elemClass = document.getElementsByClassName("container");
```

```
console.log(container);
```

Console - HTMLCollection(2)

~~console.log~~
elemClass[0].style.background = "yellow"



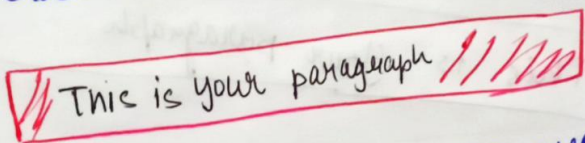
In HTML File Styles -

```
.bg-primary {  
  background: red;  
}
```

3

In js -

```
elemClass[1].classList.add("bg-primary");
```

 This is your paragraph

```
→ elemClass[1].classList.remove("bg-primary");
```

(iii) console.log(elem.innerHTML)

Click Me

```
console.log(elemClass[0].innerText)
```

Click Me

Hello!

(iv) document.getElementsByTagName

let tu = document.getElementsByTagName('div')

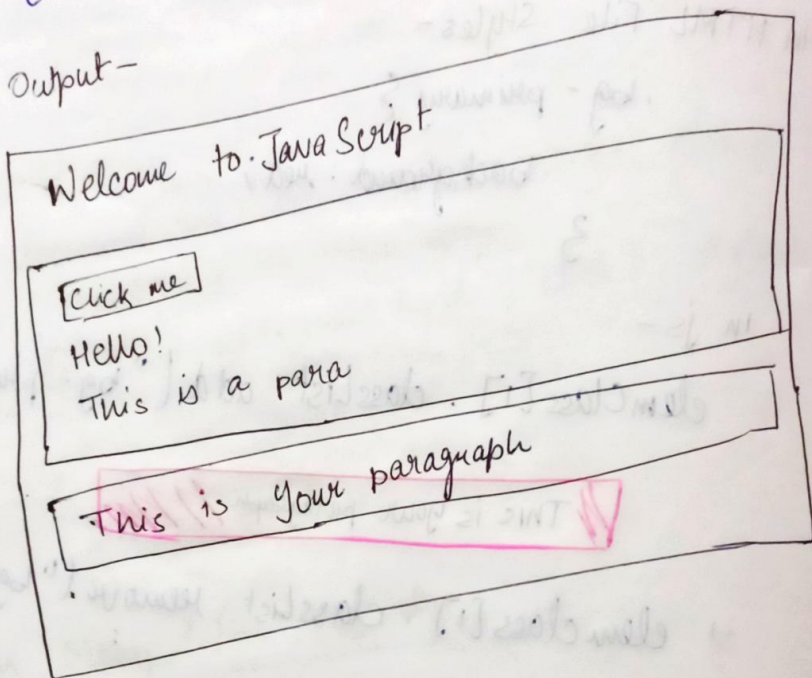
```
console.log(tu);
```

HTML Collection (2)

⇒ ~~info~~ createdElement = document.createElement('p');
createdElement.innerHTML = "This is a created para";
tn[0].appendChild(createdElement)

① →

Output -



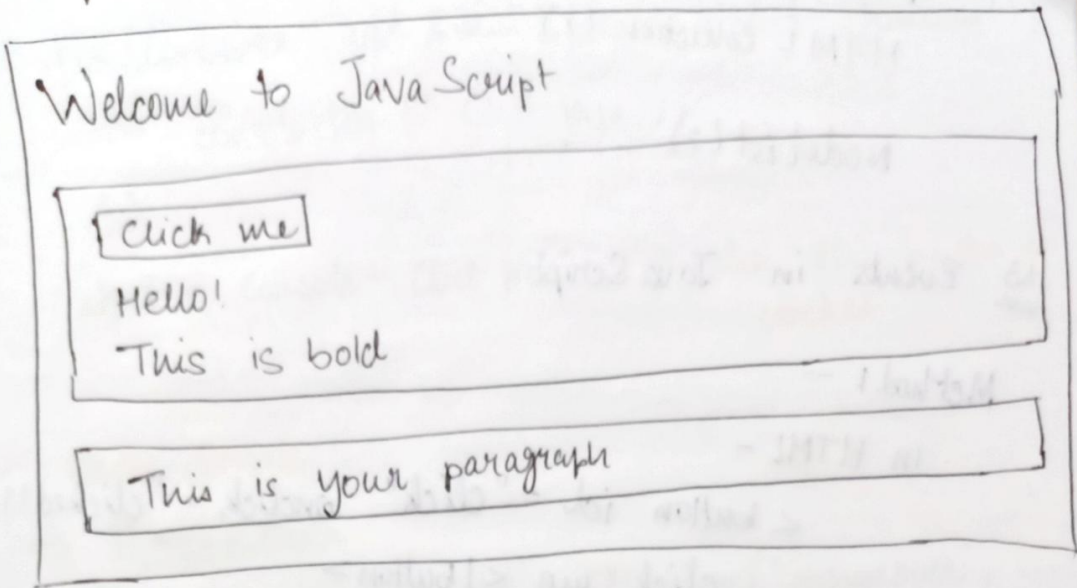
② → Replace Child -

createdElement = document.createElement('p');
createdElement.innerHTML = "This a created para";
tn[0].replaceChild(createdElement);

Output - shown above ↑

createdElement2 = document.createElement('b');
createdElement2.innerHTML = "This is bold";
tn[0].replaceChild(createdElement2, createdElement);

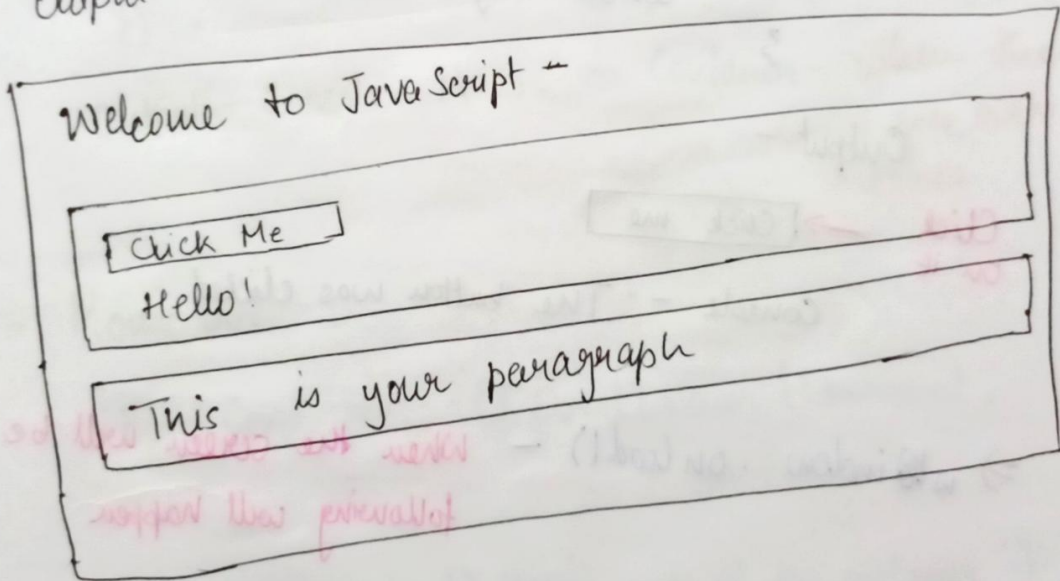
Output -



③ Remove Child -

`tn[0].removeChild(createdElement);`

Output -



(iv) selecting using Query -

`sel = document.querySelector('.container');`
`console.log(sel);`

`sel = document.querySelectorAll('.container');`
`console.log(sel);`

Output -

HTML collection (2)

NodeList (2)

13 Events in Java Script

Method 1 -

In HTML -

```
<button id = "click" onclick = "clicked()">  
click me </button>
```

In JavaScript -

```
function clicked() {
```

```
  console.log('The button was clicked');
```

```
}
```

Output -

Click
on it

→ Click me

Console - The button was clicked

⇒ `window.onload()` - When the screen will be loaded
following will happen

```
window.onload = function() {
```

```
  console.log('The totl document was loaded');
```

```
}
```

Output - Console - The document was loaded.

Method 2 -

```
First Container . addEventListener ('click', function () {  
    console.log (" Click Hwa");  
})
```

Output - Console - Click Hwa → when the button is clicked.

MORE Events

⇒ Mouse Over

```
First Container . addEventListener ('mouseover',  
    function () {
```

```
    console.log ("Mouse on Container");
```

```
    })
```

Output - Console - Mouse on Container - when the mouse goes over container.

⇒ Mouse Out

```
First Container . addEventListener ('mouseout',  
    function () {
```

```
    console.log ("Mouse out of the container");
```

```
    })
```

Output - console - Mouse out of the container

↓
when the mouse out of the container

⇒ Mouse Up

First Container . add Event Listener ('mouseup', function)

{ console.log ("Mouse up when clicked on the container");

}

Output - Console - Mouse up when clicked on the container.

↓
when the mouse is clicked

⇒ Mouse Down

First Container . add Event Listener ('mousedown',

function () {

console.log ("Mouse down when clicked on the container.");

}

Output - Console - Mouse down when clicked on the container.

↓
when the mouse is lifted.

⇒ Changing inner HTML

Type 1 -

First Container . add Event Listener ('click',

function () {

document.querySelector ("container") [0].innerHTML =

console.log ("Mouse on Container");

" We have clicked "

Output -

Welcome to JavaScript

click me
Hello!

This is your paragraph

→ This will change to ↓ when the above container is clicked.

We have clicked.

Type 2 -

```
let prevHTML = document.querySelector('.container')  
[0].innerHTML;
```

```
FirstContainer.addEventListener('mouseup', function() {
```

```
document.querySelector('.container')[0].innerHTML  
= prevHTML;
```

```
console.log("Mouse up when clicked on the container");
```

});

```
FirstContainer.addEventListener('mousedown', function() {  
document.querySelector('.container')[0].innerHTML  
= "<b> We have clicked </b>";
```

```
console.log("Mouse down when clicked on the container");
```

3)

14 Arrow Functions

```
function sum(a,b) {  
  return a+b;  
}
```

} Normal way

With Arrow Functions -

```
sum = (a,b) => {  
  return a+b;  
}
```

}

15 Set Time Out and Set Interval

```
logKaro = () => {
```

```
  console.log("set statement fixed");  
}
```

}

setTimeout (logKaro, 3000) → function → milliseconds

console - set statement fixed → after 3seconds

```
setInterval (logKaro, 4000)
```

console - set statement fixed → after every 4seconds this statement will appear.

→ To stop the interval.

```
clr = setInterval (logKaro, 4000);
```

```
clearInterval (clr);
```

↓

This will stop the statement being fixed every 4seconds.

16 JavaScript Local Storage

In console -

local storage . setItem ('name', 'harry'); → to create items
> undefined

local storage
> storage { name: 'harry', length: 1 } → to print what's in local storage

local storage . getItem ('name')

→ to see what's stored in item name

> harry

→ local storage . setItem ('name', 'harry', 'age', '18');

> undefined

local storage

> storage { name: 'harry', age: '18',
length: 2 }

use to
remove the
given item

local storage . removeItem ('name')

> undefined

local storage

> storage { age: '18', length: 1 }

removes all
the items

local storage . clear()

> undefined

local storage

> storage { length: 0 }

17 JSON - JavaScript Object Notation

⇒ It is an open standard file format which is used to exchange data.

```
obj = { name: "harry" };
```

```
json = JSON.stringify(obj);
```

```
console.log(json);
```

```
Output - { "name": "harry" }
```

To exchange data in objects is hard. That is why we use stringify function we can change object in string then easily exchange the data.

* JSON only takes double quotes

```
obj1 = { name: "harry", length: 1, a: { this: 'that' } }
```

```
json1 = JSON.stringify(obj1);
```

```
console.log(json1);
```

```
Output - { "name": "harry", "length": 1, "a": { "this": "that" } }
```

* Parsed - gives object inside objects

```
parsed = JSON.parse('{"name": "harry", "length": 1, "a": {"this": "that"}}');
```

```
console.log(parsed);
```

```
Output - { name: 'harry', length: 1, a: { this: 'that' } }
```

JavaScript for a Digital Clock

let a;

let time;

let date;

setInterval(() => {

 a = new Date();

 time = a.getHours() + ":" + a.getMinutes()
 + ":" + a.getSeconds();

 date = a.toLocaleDateString();

 document.querySelector('here').innerHTML
 = t + "
" + d;

}, 1000);

JavaScript code for Filtering Search-Bar

document.addEventListener('DOM Content Loaded', () => {

 const searchInput = document.getElementById('searchInput');

 const cards = document.querySelectorAll('.card');

 searchInput.addEventListener('input', () => {

 const query = searchInput.value.toLowerCase();

 cards.forEach(card => {

 const countryName = card.getAttribute('data-name').toLowerCase();

 if (countryName.includes(query)) {

```
card.style.display = 'flex';
else {
  card.style.display = 'none';
}
```

```
});
```

```
});
```

```
});
```

10 Event Handling

```
var btn = document.getElementById("go-button");
function buttonClicked () {
```

```
  console.log("Button Clicked");
```

```
  btn.removeEventListener("click", buttonClicked);
```

Let's the
button to be
clicked only
once

```
  btn.addEventListener("click", buttonClicked);
}
```

Output -

→ When the Button is Clicked

console - Button Clicked

11 Setting an InnerText

```

var btn = document.getElementById("go-button");
function buttonClicked() {
    console.log("Button Clicked");
    btn.removeEventListener("click", buttonClicked);
    document.getElementById("text").innerHTML =
        "Don't do it!";
    console.log(document.getElementById("text").innerHTML);
}
btn.addEventListener("click", buttonClicked);

```

12

Arrays

```

var arr1 = ["pizza", "gaming", "reading",
            "programming", "music"];
console.log(arr1);

```

Output - (5) ['pizza', 'gaming', 'reading', 'programming', 'music']

⇒ Array methods

(i) pop() - *removes the last element*

```

arr1.pop();
console.log(arr1);

```

Output - (4) ['pizza', ..., 'programming']

```

console.log(arr1.pop());

```

Output - music

If we write like this the popped element will be the output

(ii) push() - To add an item

not arr1.push("archery");

Output - (5) ['pizza', ..., 'archery']

(iii) length() - Gives the length of an array

~~arr1~~ console.log(arr1.length);

Output - 5

arr1[arr1.length - 1];

console.log(arr1);

Output - archery

⇒ arr1.forEach(function(item) {
 console.log("I like", item);
})

Output - I like pizza

I like gaming

I like reading

I like programming

I like archery

(iv) shift() - Removes the first element

Removed item
will be displayed → console.log(arr1.shift());

Output - pizza

arr1.shift()

console.log(arr1)

Output - (4) ['gaming', 'reading', 'programming', 'archery']

(v) Unshift() - Adds an item to the front

```
arr1.unshift("blacksmithing");
```

```
console.log(arr1);
```

Output - (5) ['blacksmithing', ..., 'archery']

(vi) Index ^{of} ~~Array~~() - Gives the index number of items

```
console.log(arr1.indexOf('reading'));
```

Output - 2

⇒ splice (start, end) → Removing items

```
arr1.splice(1, 2);
```

```
console.log(arr1);
```

Output - (2) ['blacksmithing', 'archery']

Note - === means comparing value and the type

```
if (arr1.indexOf('reading') > -1) {  
  console.log("I like reading");  
}
```

when something is
in array = index
when not in array
-1

Output - I like reading

```
if (arr1.indexOf('sports') === -1) {}
```

```
else {
```

```
  console.log("not in array");  
}
```

3

Output - not in array

13 Creating the text in search bar as inner HTML

```
⇒ var btn = document.getElementById("go-btn");
```

```
function buttonClicked () {
```

```
  console.log("Button clicked");
```

```
  var customText = document.getElementById("lg-input");
```

```
  var results = document.getElementById("text");
```

```
  console.log(customText.value);
```

```
  results.innerHTML = customText.value;
```

```
}
```

```
btn.addEventListener("click", buttonClicked);
```

14 Functions

```
⇒ function saySomething (phrase) {  
  console.log("He said: " + phrase);
```

```
}
```

```
saySomething("Hello how are you?")
```

Console - He said: Hello how are you?

```
⇒ function findingLength (phrase) {
```

```
  var l = phrase.length;
```

```
  return l;
```

```
}
```

```
var thisLength = findingLength("Hello this is my phrase")
```

```
console.log(thisLength);
```

Output - 23

→ Multiple parameter in functions

```
function findingLength (phrase, another) {
```

```
var l1 = phrase.length;
```

```
var l2 = another.length;
```

```
return l1 + l2;
```

```
}
```

```
var p1 = "Hello this is my phrase";
```

```
var p2 = "this is a shorter sentence";
```

```
var thisLength = findingLength(p1, p2);
```

```
console.log(thisLength);
```

Output - 49

→ If p2 is removed then ↑ this code will give error -

→ function findingLength (phrase, another) {

```
var l1 = phrase.length;
```

```
return l1;
```

```
if (typeof (another) !== and "undefined") {
```

```
var l2 = another.length;
```

```
return l1 + l2 } }
```

var p1..

Output = 49.

var p2

var thisLength = findingLength(p1)

console.log(thisLength)

Output - 23

only 1 argument
is given

15 Sorting an array

var num = [2, 4, 6, 8, 3, 1, 0]

function sortNum(a, b) {

return a-b;

}

console.log(num.sort(sortNum));

Output - [0, 1, 2, 3, 4, 6, 8]

→ logic - if $a-b$ is negative then $a < b$
so a will come before b .

if $a-b$ is positive then $a > b$
so a will come after b .

⇒

16 Exercise - Find the missing number

var Numbers = [2, 4, 6, 8, 3, 5, 1, 0]

function missingNum(Numbers) {

for (var i=0; i < numbers.length; i++) {

```

    if (num.indexOf(i) === -1) {
        return i;
    }
}
return -1;

```

```

8
console.log(missingNum(numbers));

```

Output - 7

-1 means does not exist

Logic - length of numbers

$i = 0 \rightarrow 0 < 8$

$\text{num.indexOf}(0) = 7 \neq -1$

$i = 1 \rightarrow 1 < 8$

$\text{num.indexOf}(1) = 6 \neq -1$

17 Classes

```

class Animal {

```

```

    constructor() {

```

```

        console.log("Created animal");
    }
}

```

```

3

```

```

var dog = new Animal();

```

Output - Created animal

```

var fish = new Animal();

```

Output - Created animal.

→ Creating multiple parameters in the constructor

```

class Animal {

```

```

    constructor (name, height, weight) {
        console.log ("Created animal", name);
    }

```

```

}
var dog = new Animal ("Bruno", 45, 140);
var fish = new Animal ("Goldie", 24, 100);
Output - Created animal Bruno
         Created animal Goldie

```

If another function is written inside class but outside constructor then we cannot call it. To make it work we use instance variable.

```

⇒ class Animal {
    constructor (name, height, weight) {
        this.name = name;
        this.height = height;
        this.weight = weight;
    }

```

```

    nameLength() {
        return this.name.length;
    }

```

```

}
var dog = new Animal ("Bruno", 45, 140);
console.log (dog.nameLength());

```

Output - 5

→ Static Variables

```
class Animal {
```

```
:
```

```
}
```

```
Animal.planet = "Earth";
```

codes

```
var dog = new Animal("Bruno", 45, 140);
```

```
console.log(dog.constructor.planet);
```

Outputs - Earth

16 Extended Classes

```
class Animal { ... }
```

```
class dog extends Animal {
```

```
constructor(name, height, weight, barkVolume,  
leashColor, speed) {
```

```
super(name, height, weight);
```

```
this.barkVolume = barkVolume;
```

```
this.leashColor = leashColor;
```

```
this.speed = speed
```

```
bark() {
```

```
if (this.barkVolume > 50) {
```

```
console.log(this.name, "barks loudly",  
"Volume:", this.barkVolume);
```

```
}
```

```
else {
```

```
console.log (this.name, "barks loudly", "volume",  
this.barkVolume);
```

```
class fish extends Animal {  
  constructor (name, height, weight, swimSpeed) {  
    super (name, height, weight);  
    this.swimSpeed = swimSpeed;
```

```
  }  
  swim (swim) {  
    if (this.swimSpeed > 50) {  
      console.log (this.name, "swims fastly", "speed",  
this.swimSpeed);
```

```
    }  
    else {  
      console.log (this.name, "swims slowly", "speed",  
this.swimSpeed);
```

```
  }  
  swimming (chaserSpeed) {
```

```
    if (this.swimSpeed > chaserSpeed) {  
      console.log (this.name, "got away");
```

```
    }
```

else {
 console.log(this.name, " was caught");
}

}

}

}

var king = new dog("Pluto", 67, 34, 45, "red", 50);

king.bark();

Output - Pluto barks timidly volume: 45

var shark = new fish("Lovely", 78, 56, 67);

shark.swim();

Output - Lovely swims fastly speed: 67

shark.swimming(king.speed);

Output - Lovely got away